

Documentação Técnica - TP2 FPAA - Tango Puzzle

1. Modelagem do Problema

O problema foi implementado em Java. A ideia principal foi representar o tabuleiro como uma matriz de células, onde cada célula pode ter um dos três valores possíveis: Sol, Lua ou vazio.

No código, o tabuleiro fica na classe **Board**. Ela guarda:

- o tamanho do tabuleiro
- uma matriz de objetos **Cell**
- uma lista de restrições entre células vizinhas.

Cada célula fica na classe **Cell**, que possui linha, coluna, valor e uma informação dizendo se ela era fixa desde o início do jogo. Isso é importante porque as células que já vêm preenchidas no arquivo de entrada não podem ser modificadas pelos algoritmos. Na prática, o método que altera uma célula bloqueia essa modificação quando a célula é fixa, então os algoritmos só conseguem trabalhar sobre as posições que estavam vazias no arquivo.

Os valores das células foram representados pelo enum **CellValue**, com as opções:

- **SUN**, para Sol
- **MOON**, para Lua
- **EMPTY**, para célula vazia.

As restrições de igualdade e oposição foram representadas pela classe **Constraint**. Cada restrição guarda as coordenadas de duas células e o tipo da restrição. O tipo fica no enum **ConstraintType**, com os valores:

- **EQUAL**, quando as duas células devem ter o mesmo símbolo
- **OPPOSITE**, quando as duas células devem ter símbolos diferentes.

A leitura do tabuleiro é feita pela classe **BoardReader**. O arquivo de entrada começa com o tamanho do tabuleiro, depois vem a grade inicial e depois as restrições. Na grade, usamos **S** para Sol, **L** para Lua e **.** para vazio. Nas restrições, usamos **H** para restrição horizontal e **V** para restrição vertical. O símbolo **=** indica igualdade e **X** indica oposição.

A impressão do tabuleiro é feita pela classe **BoardPrinter**, que mostra tanto as células quanto os sinais de restrição no terminal.

As regras do jogo foram separadas em classes próprias, usando a interface **Rule**. Essa separação ajuda porque as mesmas validações são usadas tanto na força bruta quanto no backtracking. As classes principais de validação são:

- **AdjacencyRule**, que impede três símbolos iguais seguidos na horizontal ou vertical
- **BalanceRule**, que verifica se cada linha e coluna possui a mesma quantidade de sóis e luas
- **ConstraintRule**, que verifica as restrições de igualdade e oposição.

A classe **RuleValidator** junta essas regras e oferece dois tipos de validação: uma validação parcial, usada durante o backtracking, e uma validação completa, usada quando o tabuleiro já está preenchido. Na validação parcial, o código não precisa reavaliar tudo sempre: para adjacência e balanceamento ele olha a linha e a coluna da célula recém-preenchida, e para as restrições ele confere apenas aquelas ligadas a essa célula. A validação completa, por outro lado, percorre todas as linhas, colunas e restrições.

No programa principal, o arquivo do puzzle é lido novamente antes de executar cada algoritmo. Fizemos isso porque tanto a força bruta quanto o backtracking alteram o tabuleiro em memória enquanto procuram a solução. Assim, os dois métodos começam sempre do mesmo estado inicial, o que deixa a comparação mais justa.

2. Estratégia de Resolução por Força Bruta

Na força bruta, o algoritmo não tenta ser inteligente durante o preenchimento. Ele primeiro identifica todas as células vazias do tabuleiro e depois percorre as combinações possíveis para essas células até encontrar uma solução válida ou esgotar o espaço de busca.

Como cada célula vazia pode receber apenas dois valores, Sol ou Lua, se existirem **k** células vazias, o total de combinações possíveis é:

$$2^k$$

No código, isso fica na classe **BruteForceSolver**. O algoritmo monta uma lista com as posições vazias e usa uma máscara numérica para representar uma tentativa de preenchimento. Para cada número da máscara, cada bit representa o valor de uma célula vazia. Quando o bit é 0, a célula recebe Sol. Quando o bit é 1, a célula recebe Lua.

Por exemplo, se existirem 4 células vazias, o algoritmo testa todas as combinações de **0000** até **1111**, totalizando 16 possibilidades.

Depois que uma combinação completa é colocada no tabuleiro, o algoritmo chama o validador para verificar se o tabuleiro inteiro respeita todas as regras:

- todas as células precisam estar preenchidas
- não pode haver três símbolos iguais seguidos
- cada linha deve ter a mesma quantidade de sóis e luas
- cada coluna deve ter a mesma quantidade de sóis e luas
- as restrições = e X precisam estar corretas.

Se a combinação for válida, o algoritmo termina ali e o tabuleiro fica resolvido. Por isso, em um caso com solução, ele não necessariamente testa todas as combinações possíveis. Ele só testaria todas se a solução estivesse na última combinação ou se nenhuma solução fosse encontrada. Se a combinação atual não for válida, ele passa para a próxima máscara.

Essa estratégia funciona bem em tabuleiros pequenos, como o 4x4 usado nos testes. Porém, em tabuleiros maiores, o número de combinações cresce muito rápido. Por isso a força bruta foi usada principalmente para demonstrar o espaço total de busca e comparar com o backtracking.

3. Estratégia de Resolução por Backtracking

O backtracking também preenche o tabuleiro por tentativa, mas ele não espera o tabuleiro inteiro ficar pronto para validar. A cada valor colocado, ele verifica se aquela escolha ainda pode levar a uma solução válida.

No código, o backtracking fica na classe **BacktrackingSolver**. A função principal procura a próxima célula vazia do tabuleiro, percorrendo a matriz de cima para baixo e da esquerda para a direita. Mantivemos essa escolha simples porque ela já era suficiente para os tabuleiros usados no trabalho. Uma heurística mais elaborada, como procurar primeiro a célula com mais restrições, poderia reduzir algumas tentativas, mas também deixaria o código mais difícil de acompanhar. Como o objetivo era comparar força bruta e backtracking, preferimos deixar a poda das regras como o ponto principal da implementação.

Depois de escolher uma célula vazia, o algoritmo tenta preencher primeiro com Sol e depois com Lua. Essa ordem não muda as regras do problema, mas pode mudar a quantidade de nós explorados. Se a solução estiver mais próxima das escolhas que começam com Sol, o algoritmo chega nela mais rápido. Se várias escolhas iniciais com Sol levarem a caminhos errados, ele precisa voltar mais vezes antes de tentar Lua.

A condição de parada acontece quando não existe mais nenhuma célula vazia. Nesse caso, o tabuleiro está completo e o algoritmo faz uma validação final usando todas as regras. Se o tabuleiro completo for válido, a solução foi encontrada.

Durante a recursão, depois de colocar um valor em uma célula, o algoritmo chama a validação parcial. Essa validação já elimina escolhas ruins antes de continuar, sem esperar o tabuleiro inteiro ficar completo. Por exemplo:

- se aparecerem três sóis seguidos, aquele caminho é abandonado
- se uma linha tiver mais sóis ou luas do que o permitido, aquele caminho é abandonado
- se uma restrição de igualdade ou oposição já estiver violada, aquele caminho é abandonado.

Essas verificações são feitas sobre a parte do tabuleiro afetada pela última escolha: linha, coluna e restrições da célula preenchida. Isso foi uma decisão simples, mas importante, porque evita repetir a validação completa em cada nível da recursão.

Quando uma tentativa não serve, o algoritmo desfaz a escolha, coloca a célula novamente como vazia e volta para tentar outro valor. Esse processo é o retrocesso. No código, também foram guardadas estatísticas simples de execução, como quantidade de nós explorados e quantidade de retrocessos, para facilitar a comparação com a força bruta.

A diferença principal em relação à força bruta é que o backtracking evita testar muitas combinações completas que já estavam erradas desde o meio do preenchimento. Por isso ele é muito mais rápido nos tabuleiros 6x6.

4. Exemplos de Execução

Para compilar o projeto, foi usado:

```
javac -d out $(find src -name '*.java')
```

Nas execuções, usamos as estatísticas para comparar os algoritmos porque o tempo em milissegundos pode variar de uma máquina para outra e até entre duas execuções seguidas. Já a quantidade de combinações testadas, nós explorados e retrocessos mostra melhor o comportamento da busca para aquele tabuleiro.

Exemplo 1 - 4x4 fácil com força bruta e backtracking

Comando:

```
java -cp out Main puzzles/easy_4x4.txt ambos
```

Recorte da execução:

```
=== Tabuleiro Inicial ===
+-----+-----+-----+-----+
| S  =  .  | L  | .  |
+-----+-----+---X---+-----+
| .  | L  x  .  | .  |
+-----+-----+-----+-----+
| S  | .  | .  x  .  |
+---X---+-----+-----+-----+
| .  | S  | .  | S  |
+-----+-----+-----+-----+

===== FORCA BRUTA =====
=== Tabuleiro Resolvido ===
+-----+-----+-----+-----+
| S  =  S  | L  | L  |
+-----+-----+---X---+-----+
| L  | L  x  S  | S  |
+-----+-----+-----+-----+
| S  | L  | S  x  L  |
+---X---+-----+-----+-----+
| L  | S  | L  | S  |
+-----+-----+-----+-----+

--- Estatisticas da Forca Bruta ---
Combinacoes testadas : 935
Total possivel       : 1024 (2^10)
Tempo total          : 19 ms

===== BACKTRACKING =====
=== Tabuleiro Resolvido ===
+-----+-----+-----+-----+
| S  =  S  | L  | L  |
+-----+-----+---X---+-----+
| L  | L  x  S  | S  |
+-----+-----+-----+-----+
| S  | L  | S  x  L  |
+---X---+-----+-----+-----+
| L  | S  | L  | S  |
+-----+-----+-----+-----+

--- Estatisticas do Backtracking ---
Nos explorados : 16
Retrocessos    : 6
Tempo total     : 4 ms
```

Nesse exemplo, a força bruta ainda é viável porque existem apenas 10 células vazias, gerando um espaço de 1024 combinações possíveis. A solução foi encontrada depois de 935 tentativas, ou seja, antes de percorrer o espaço inteiro.

Exemplo 2 - 6x6 fácil com backtracking

Comando:

```
java -cp out Main puzzles/easy_6x6.txt backtracking
```

Recorte da execução:

```
=== Tabuleiro Inicial ===
+-----+-----+-----+-----+
| S  =  .  | .  | .  | .  | .  |
+-----+-----+-----+-----+
| .  | .  | L  | .  | .  | .  |
+-----+-----+-----+-----+
| .  | .  | .  | x  | .  | .  |
+-----+-----+-----+-----+
| .  | .  | .  | .  | .  | .  |
+-----+-----+-----+-----+
| .  | .  | .  | .  | L  | .  |
+-----+-----+-----+-----+
| .  | .  | .  | .  | .  | .  |
+-----+-----+-----+-----+

===== BACKTRACKING =====
=== Tabuleiro Resolvido ===
+-----+-----+-----+-----+
| S  =  S  | L  | S  | L  | L  |
+-----+-----+-----+-----+
| S  | S  | L  | S  | L  | L  |
+-----+-----+-----+-----+
| L  | L  | S  x  L  | S  | S  |
+-----+-----+-----+-----+
| S  | S  | L  | L  | S  | L  |
+-----+-----+-----+-----+
| L  | L  | S  | S  | L  | S  |
+-----+-----+-----+-----+
| L  | L  | S  | L  | S  | S  |
+-----+-----+-----+-----+

--- Estatísticas do Backtracking ---
Nos explorados : 51
Retrocessos    : 18
Tempo total    : 16 ms
```

Nesse caso, a força bruta não foi usada porque o tabuleiro 6x6 possui muito mais células vazias. Mesmo sendo fácil, o número de combinações possíveis já fica muito alto para uma demonstração prática em sala ou no terminal. Os tempos em milissegundos podem variar de uma execução para outra, mas os contadores de nós e retrocessos mostram bem a diferença entre os métodos.

Exemplo 3 - 6x6 médio com backtracking

Comando:

```
java -cp out Main puzzles/medium_6x6.txt backtracking
```

Recorte da execução:

```
=== Tabuleiro Inicial ===
+---+---+---+---+---+---+
| L x . | . | . = . | L |
+---X---+---+---+---+---+
| . | . | S | . = . | . |
+---+---+---+---+---+---+
| . x . | . | L | . | . |
+---+---+---+---X---+---+
| . | L = . | . | . | . |
+---+---+---+---+---X---+
| . | . | . = L | . | . |
+---+---+---+---+---X---+
| . x . | . = . | L | . |
+---+---+---+---+---+---+

===== BACKTRACKING =====
=== Tabuleiro Resolvido ===
+---+---+---+---+---+---+
| L x S | L | S = S | L |
+---X---+---+---+---+---+
| S | L | S | L = L | S |
+---+---+---+---+---+---+
| L x S | S | L | S | L |
+---+---+---+---X---+---+
| S | L = L | S | L | S |
+---+---+---+---+---X---+
| L | S | L = L | S | S |
+---+---+---+---+---X---+
| S x L | S = S | L | L |
+---+---+---+---+---+---+

--- Estatísticas do Backtracking ---
Nos explorados : 83
Retrocessos    : 54
Tempo total    : 12 ms
```

Esse exemplo tem mais restrições e exigiu mais nós explorados e mais retrocessos do que o 6x6 fácil. Isso mostra que o algoritmo realmente faz tentativas e volta quando encontra caminhos inválidos, em vez de simplesmente aplicar uma sequência fixa de deduções.

5. Análise de Complexidade

O ponto principal da complexidade do Tango é o número de células vazias. Se o tabuleiro possui k células vazias, cada uma delas pode receber dois valores. Então, no pior caso, o espaço de busca tem tamanho:

2^k

Na força bruta, esse é o comportamento principal. O algoritmo percorre as combinações possíveis e só depois valida o tabuleiro completo. Por isso, no pior caso, a complexidade de busca é exponencial. Além disso, cada combinação testada precisa ser validada, verificando linhas, colunas e restrições. Mesmo assim, o maior problema continua sendo a quantidade de combinações.

No exemplo 4x4, existiam 10 células vazias. Então o total possível era:

$2^{10} = 1024$

Isso ainda é pequeno. Mas no 6x6 fácil existem 33 células vazias, então o total teórico seria:

$$2^{33} = 8589934592$$

Esse número já passa de 8 bilhões de combinações. Por isso a força bruta deixa de ser uma opção prática.

O backtracking também tem pior caso exponencial, porque em uma situação muito ruim ele ainda poderia precisar tentar muitas combinações. A diferença é que ele usa poda. Em vez de esperar o tabuleiro todo estar preenchido, ele verifica as regras a cada passo, logo após preencher uma célula.

As principais podas usadas foram:

- não permitir três símbolos iguais seguidos
- não deixar uma linha ou coluna passar da metade de sóis ou luas
- verificar as restrições de igualdade e oposição assim que as células envolvidas já possuem valor.

Essas podas diminuem muito o número de caminhos explorados. No 4x4, por exemplo, a força bruta tinha 1024 combinações possíveis, mas o backtracking explorou apenas 16 nós. Nos tabuleiros 6x6, o backtracking também resolveu rapidamente, mesmo quando a força bruta teria um espaço de busca muito grande.

Portanto, a força bruta é importante para mostrar o problema completo e a explosão combinatória. Já o backtracking é a estratégia mais adequada para resolver o puzzle na prática, porque aproveita as regras do jogo para cortar caminhos inválidos antes de terminar o preenchimento.